

Introduction: Why Study Elementary Sorts?

Sorting is one of the most fundamental operations in computer science. Before studying advanced algorithms such as Merge Sort or Quick Sort, it is essential to understand the simpler elementary sorting algorithms. They are easy to reason about, straightforward to implement, and provide the conceptual foundation for analysing time complexity.

This tutorial covers:

1. **Selection Sort** — find the minimum, place it in position.
2. **Insertion Sort** — build a sorted prefix one element at a time.
3. **Time Complexity Analysis** — best, average, and worst case for both.

Part 1: Selection Sort

1.1 Core Idea

How Selection Sort Works

Selection sort divides the array into two logical regions:

- **Sorted region** (left) — elements already in their final position.
- **Unsorted region** (right) — elements not yet placed.

On each **pass** i (from 0 to $n - 2$):

1. Scan the entire unsorted region to find the **minimum** element.
2. **Swap** that minimum with the element at position i .
3. The sorted region grows by one element.

After $n - 1$ passes the entire array is sorted.

1.2 Step-by-Step Trace

Trace: sort [64, 25, 12, 22, 11]

Pass	[0]	[1]	[2]	[3]	[4]	Action
Start	64	25	12	22	11	Initial array
$i = 0$	11	25	12	22	64	Min=11 (pos 4) → swap with pos 0
$i = 1$	11	12	25	22	64	Min=12 (pos 2) → swap with pos 1
$i = 2$	11	12	22	25	64	Min=22 (pos 3) → swap with pos 2
$i = 3$	11	12	22	25	64	Min=25 (pos 3) → no swap needed
Done	11	12	22	25	64	Array sorted

Sorted region
 Current pass (minimum shown in **bold**)

1.3 Implementation

Listing 1: Selection Sort — complete implementation with trace output

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  // Print the array with a label
6  void printArray(const string& label, const vector<int>& arr) {
7      cout << label << ": [ ";
8      for (int x : arr) cout << x << " ";
9      cout << "]\n";
10 }
11
12 // Selection Sort
13 void selectionSort(vector<int>& arr) {
14     int n = arr.size();
15
16     for (int i = 0; i < n - 1; i++) {
17         // Find index of minimum in unsorted region [i .. n-1]
18         int minIdx = i;
19         for (int j = i + 1; j < n; j++) {
20             if (arr[j] < arr[minIdx])
21                 minIdx = j;
22         }
23
24         // Swap minimum into its correct position
25         if (minIdx != i)
26             swap(arr[i], arr[minIdx]);
27
28         // Show state after this pass
29         cout << "Pass " << i << " (min=" << arr[i] << " placed): ";
30         printArray("", arr);
31     }
32 }
33

```

```
34 int main() {
35     vector<int> data = {64, 25, 12, 22, 11};
36
37     printArray("Before", data);
38     cout << "\n";
39     selectionSort(data);
40     cout << "\n";
41     printArray("After ", data);
42 }
```

Expected output:

Before: [64 25 12 22 11]

Pass 0 (min=11 placed): : [11 25 12 22 64]

Pass 1 (min=12 placed): : [11 12 25 22 64]

Pass 2 (min=22 placed): : [11 12 22 25 64]

Pass 3 (min=25 placed): : [11 12 22 25 64]

After : [11 12 22 25 64]

1.4 Generic Template Version

Listing 2: Selection Sort — generic template (works on any comparable type)

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5
6  template <typename T>
7  void selectionSort(vector<T>& arr) {
8      int n = arr.size();
9      for (int i = 0; i < n - 1; i++) {
10         int minIdx = i;
11         for (int j = i + 1; j < n; j++)
12             if (arr[j] < arr[minIdx]) minIdx = j;
13         if (minIdx != i) swap(arr[i], arr[minIdx]);
14     }
15 }
16
17 template <typename T>
18 void print(const vector<T>& arr) {
19     for (const T& x : arr) cout << x << " ";
20     cout << "\n";
21 }
22
23 int main() {
24     vector<int>      ints      = {5, 2, 8, 1, 9, 3};
25     vector<double> doubles = {3.14, 1.41, 2.72, 0.57};
26     vector<string> words  = {"banana", "apple", "cherry", "date"};
```

```
27  
28     selectionSort(ints);    print(ints);  
29     selectionSort(doubles); print(doubles);  
30     selectionSort(words);  print(words);  
31 }
```

Expected output:

1 2 3 5 8 9

0.57 1.41 2.72 3.14

apple banana cherry date

1.5 Key Properties of Selection Sort

Selection Sort — Properties at a Glance

Comparisons	Always $\frac{n(n-1)}{2}$, regardless of input order.
Swaps	At most $n - 1$ swaps — the fewest of any elementary sort.
Stability	Not stable by default (a swap can move equal elements past each other).
In-place	Yes — uses only $O(1)$ extra memory.
Best for	Situations where the <i>cost of writing</i> is very high and comparisons are cheap (e.g. flash memory).

Part 2: Insertion Sort

2.1 Core Idea

How Insertion Sort Works

Insertion sort mimics the way a card player sorts a hand of cards:

- **Sorted region** (left) — always kept in order.
- **Key element** — the next element taken from the unsorted region.

On each **pass** i (from 1 to $n - 1$):

1. Pick element at position i as the **key**.
2. **Shift** all sorted-region elements that are *greater than* the key one position to the right.
3. **Insert** the key into the gap that was created.

Unlike selection sort, insertion sort can terminate a pass early if it finds an element $\leq$ key — this is what gives it $O(n)$ best-case performance on nearly sorted data.

2.2 Step-by-Step Trace

Trace: sort [12, 11, 13, 5, 6]

Pass	Key	[0]	[1]	[2]	[3]	[4]	Action
Start	—	12	11	13	5	6	Initial array
$i = 1$	11	12	11	13	5	6	$12 > 11$: shift right; insert 11 at [0]
$i = 2$	13	11	12	13	5	6	$12 < 13$: stop; 13 stays at [2]
$i = 3$	5	11	12	13	5	6	All three shift right; insert 5 at [0]
$i = 4$	6	5	11	12	13	6	11,12,13 shift; insert 6 at [1]
Done	—	5	6	11	12	13	Array sorted

Sorted region

Elements shifted right

Current pass (key in **bold**)

2.3 Implementation

Listing 3: Insertion Sort — complete implementation with trace output

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 void printArray(const string& label, const vector<int>& arr) {
6     cout << label;
7     for (int x : arr) cout << x << " ";
8 }
```

```
8     cout << "\n";
9 }
10
11 void insertionSort(vector<int>& arr) {
12     int n = arr.size();
13
14     for (int i = 1; i < n; i++) {
15         int key = arr[i];    // element to be inserted
16         int j = i - 1;      // start of the shifting scan
17
18         // Shift elements greater than 'key' one position to the right
19         while (j >= 0 && arr[j] > key) {
20             arr[j + 1] = arr[j];
21             j--;
22         }
23
24         arr[j + 1] = key;    // insert key into the correct gap
25
26         cout << "Pass " << i << " (key=" << key << "): ";
27         printArray("", arr);
28     }
29 }
30
31 int main() {
32     vector<int> data = {12, 11, 13, 5, 6};
33
34     printArray("Before: ", data);
35     cout << "\n";
36     insertionSort(data);
37     cout << "\n";
38     printArray("After:  ", data);
39 }
```

Expected output:

Before: 12 11 13 5 6

Pass 1 (key=11): 11 12 13 5 6

Pass 2 (key=13): 11 12 13 5 6

Pass 3 (key=5): 5 11 12 13 6

Pass 4 (key=6): 5 6 11 12 13

After: 5 6 11 12 13

2.4 Best-Case Demonstration: Nearly Sorted Input

Listing 4: Insertion Sort — counting comparisons on best vs worst case

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
```

```

4
5 long long insertionSortCount(vector<int> arr) {
6     long long comparisons = 0;
7     int n = arr.size();
8     for (int i = 1; i < n; i++) {
9         int key = arr[i], j = i - 1;
10        while (j >= 0 && (++comparisons, arr[j] > key)) {
11            arr[j + 1] = arr[j];
12            j--;
13        }
14        arr[j + 1] = key;
15    }
16    return comparisons;
17 }
18
19 int main() {
20     int n = 8;
21
22     // Already sorted: best case  $O(n)$ 
23     vector<int> best = {1, 2, 3, 4, 5, 6, 7, 8};
24     // Reverse sorted: worst case  $O(n^2)$ 
25     vector<int> worst = {8, 7, 6, 5, 4, 3, 2, 1};
26     // Random: average case
27     vector<int> avg = {4, 2, 7, 1, 8, 3, 6, 5};
28
29     cout << "n = " << n << "\n";
30     cout << "Best case (sorted) comparisons: " << insertionSortCount(best) << "\n";
31     cout << "Avg case (random) comparisons: " << insertionSortCount(avg) << "\n";
32     cout << "Worst case (reversed) comparisons: " << insertionSortCount(worst) << "\n";
33     cout << "Theoretical worst  $n*(n-1)/2 =$ " << (long long)n*(n-1)/2 << "\n";
34 }

```

Expected output:

n = 8

Best case (sorted) comparisons: 7

Avg case (random) comparisons: 14

Worst case (reversed) comparisons: 28

Theoretical worst $n*(n-1)/2 = 28$

2.5 Generic Template Version

Listing 5: Insertion Sort — generic template with custom comparator

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <functional>
5 using namespace std;
6
7 // Generic insertion sort with a custom comparator (default: ascending)
8 template <typename T, typename Compare = less<T>>

```

```
9 void insertionSort(vector<T>& arr, Compare cmp = Compare{}) {
10     int n = arr.size();
11     for (int i = 1; i < n; i++) {
12         T key = arr[i];
13         int j = i - 1;
14         while (j >= 0 && cmp(key, arr[j])) {
15             arr[j + 1] = arr[j];
16             j--;
17         }
18         arr[j + 1] = key;
19     }
20 }
21
22 template <typename T>
23 void print(const vector<T>& arr) {
24     for (const T& x : arr) cout << x << " ";
25     cout << "\n";
26 }
27
28 int main() {
29     // Sort integers ascending (default)
30     vector<int> v1 = {5, 2, 8, 1, 9};
31     insertionSort(v1);
32     cout << "Ascending: "; print(v1);
33
34     // Sort integers descending using greater<int>
35     vector<int> v2 = {5, 2, 8, 1, 9};
36     insertionSort(v2, greater<int>{});
37     cout << "Descending: "; print(v2);
38
39     // Sort strings by length (shortest first)
40     vector<string> words = {"banana", "kiwi", "fig", "apple", "pear"};
41     insertionSort(words, [](const string& a, const string& b){
42         return a.size() < b.size();
43     });
44     cout << "By length: "; print(words);
45 }
```

Expected output:

Ascending: 1 2 5 8 9

Descending: 9 8 5 2 1

By length: fig kiwi pear apple banana

2.6 Key Properties of Insertion Sort

Insertion Sort — Properties at a Glance

Comparisons	Best: $O(n)$ Average/Worst: $O(n^2)$
Swaps/Shifts	At most $O(n^2)$ shifts; each element moves at most once per pass.
Stability	Stable — equal elements preserve their original relative order.
In-place	Yes — $O(1)$ extra memory.
Adaptive	Yes — runs in $O(n)$ on already-sorted or nearly-sorted input.
Best for	Small arrays, nearly sorted data, or as the base case inside Merge Sort / Tim Sort.

Part 3: Time Complexity Analysis

3.1 What is Time Complexity?

Big-O Notation — A Quick Refresher

Time complexity measures how the number of *basic operations* (comparisons, swaps, assignments) grows as the input size n increases. We express this using **Big-O notation**, which captures the *dominant term* while ignoring constants.

Notation	Name	Example
$O(1)$	Constant	Array access
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Single loop
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Nested loop
$O(2^n)$	Exponential	Recursive subsets

We distinguish three scenarios:

- **Best case** (Ω) — the most favourable input (e.g. already sorted).
- **Worst case** (O) — the most costly input (e.g. reverse sorted).
- **Average case** (Θ) — expected behaviour over all possible inputs.

3.2 Analysis of Selection Sort

Counting Operations in Selection Sort

On an array of size n , the outer loop runs $n - 1$ times. On pass i , the inner loop runs $n - 1 - i$ times.

Total comparisons:

$$\sum_{i=0}^{n-2} (n - 1 - i) = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2} = O(n^2)$$

Critical observation: the inner loop *always* runs to completion — it never terminates early. Therefore:

Case	Comparisons	Swaps	Overall
Best (sorted)	$n(n - 1)/2$	0	$O(n^2)$
Average (random)	$n(n - 1)/2$	$O(n)$	$O(n^2)$
Worst (reversed)	$n(n - 1)/2$	$n - 1$	$O(n^2)$

Selection sort has **identical** best, average, and worst-case complexities. It is **not adaptive**.

3.3 Analysis of Insertion Sort

Counting Operations in Insertion Sort

On pass i , the inner **while** loop runs between 1 and i times, depending on how many elements are greater than the key.

Worst case (reverse sorted — every element must shift all the way):

$$\sum_{i=1}^{n-1} i = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = O(n^2)$$

Best case (already sorted — inner loop exits after 1 comparison per pass):

$$\sum_{i=1}^{n-1} 1 = n - 1 = O(n)$$

Case	Comparisons	Shifts	Overall
Best (sorted)	$n - 1$	0	$O(n)$
Average (random)	$\sim n^2/4$	$\sim n^2/4$	$O(n^2)$
Worst (reversed)	$n(n-1)/2$	$n(n-1)/2$	$O(n^2)$

Insertion sort is **adaptive** — it takes advantage of existing order in the input.

3.4 Empirical Comparison

Listing 6: Empirical comparison: counting operations for both sorts

```

1 #include <iostream>
2 #include <vector>
3 #include <numeric>    // iota
4 #include <algorithm>  // reverse
5 using namespace std;
6
7 struct Stats { long long comparisons; long long swapsOrShifts; };
8
9 Stats selectionSortStats(vector<int> arr) {
10     int n = arr.size();
11     Stats s{0, 0};
12     for (int i = 0; i < n - 1; i++) {
13         int minIdx = i;
14         for (int j = i + 1; j < n; j++) {
15             s.comparisons++;
16             if (arr[j] < arr[minIdx]) minIdx = j;
17         }
18         if (minIdx != i) { swap(arr[i], arr[minIdx]); s.swapsOrShifts++; }
19     }
20     return s;
21 }
22

```

```

23 Stats insertionSortStats(vector<int> arr) {
24     int n = arr.size();
25     Stats s{0, 0};
26     for (int i = 1; i < n; i++) {
27         int key = arr[i], j = i - 1;
28         while (j >= 0 && (s.comparisons++, arr[j] > key)) {
29             arr[j + 1] = arr[j]; j--; s.swapsOrShifts++;
30         }
31         if (arr[j + 1] != key) arr[j + 1] = key;
32     }
33     return s;
34 }
35
36 void runTest(const string& label, vector<int> arr) {
37     cout << "\n--- " << label << " (n=" << arr.size() << ") ---\n";
38     auto ss = selectionSortStats(arr);
39     auto is = insertionSortStats(arr);
40     cout << "Selection: " << ss.comparisons << " cmp, "
41           << ss.swapsOrShifts << " swaps\n";
42     cout << "Insertion: " << is.comparisons << " cmp, "
43           << is.swapsOrShifts << " shifts\n";
44 }
45
46 int main() {
47     int n = 10;
48     vector<int> sorted(n), reversed(n), random_ = {3,1,4,1,5,9,2,6,5,3};
49     iota(sorted.begin(), sorted.end(), 1);
50     iota(reversed.begin(), reversed.end(), 1);
51     reverse(reversed.begin(), reversed.end());
52
53     runTest("Already sorted", sorted);
54     runTest("Reverse sorted", reversed);
55     runTest("Random", random_);
56 }

```

Expected output:

--- Already sorted (n=10) ---

Selection: 45 cmp, 0 swaps

Insertion: 9 cmp, 0 shifts

--- Reverse sorted (n=10) ---

Selection: 45 cmp, 9 swaps

Insertion: 45 cmp, 45 shifts

--- Random (n=10) ---

Selection: 45 cmp, 7 swaps

Insertion: 21 cmp, 17 shifts

3.5 Growth Rate Visualisation

Listing 7: Printing a growth-rate table for $n \in \{100, 500, 1000, 5000, 10000\}$

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     long long sizes[] = {100, 500, 1000, 5000, 10000};
7
8     cout << setw(8) << "n"
9         << setw(18) << "n*(n-1)/2 (Sel/Ins worst)"
10        << setw(10) << "n-1 (Ins best)" << "\n";
11     cout << string(36, '-') << "\n";
12
13     for (long long n : sizes) {
14         cout << setw(8) << n
15             << setw(18) << n*(n-1)/2
16             << setw(14) << n-1 << "\n";
17     }
18 }
```

Expected output:

n	n*(n-1)/2 (Sel/Ins worst)	n-1 (Ins best)
100	4950	99
500	124750	499
1000	499500	999
5000	12497500	4999
10000	49995000	9999

3.6 Side-by-Side Comparison

Selection Sort vs. Insertion Sort — Full Comparison

Property	Selection Sort	Insertion Sort
Best-case time	$O(n^2)$	$O(n)$
Average-case time	$O(n^2)$	$O(n^2)$
Worst-case time	$O(n^2)$	$O(n^2)$
Space complexity	$O(1)$	$O(1)$
Stable	No	Yes
Adaptive	No	Yes
Number of swaps	$O(n)$ (at most $n - 1$)	$O(n^2)$ shifts
Performance on sorted input	Same as random	Much faster
Preferred use case	Minimise writes	Nearly sorted / small n

3.7 Practical Guidelines

When to Use Each Algorithm

- **Use Insertion Sort** when:
 - The array is **small** ($n \leq 20$) — constant factors dominate.
 - The data is **nearly sorted** (few elements out of place).
 - You need a **stable sort** with no extra memory.
 - It is used as the **base case** inside Merge Sort or Tim Sort.
- **Use Selection Sort** when:
 - The **cost of writing** to memory is very high (e.g. EEPROM, flash).
 - You need a guaranteed small number of **swaps**.
 - Simplicity of implementation is valued over raw performance.
- **Avoid both** for large n (prefer $O(n \log n)$ algorithms):
 - `std::sort` (typically Intro Sort: Quick Sort + Heap Sort + Insertion Sort).
 - Merge Sort when stability is required.
 - Tim Sort (used in Python, Java) — hybrid Insertion + Merge Sort.

Lab Exercises

Exercise 1
[**Selection Sort — Trace**] Given the array [29, 10, 14, 37, 13], manually trace all passes of selection sort. For each pass write: the index of the minimum found, the swap performed, and the state of the array after the swap.

Then implement the sort, run it, and verify your manual trace matches the output.

Exercise 2
[**Selection Sort — Descending Order**] Modify selection sort to sort in **descending** order by finding the *maximum* element on each pass instead of the minimum. Test with `vector<int> v = {3, 6, 1, 8, 2, 9, 4}`.

Exercise 3
[**Insertion Sort — Trace**] Given the array [8, 3, 7, 1, 5], trace all passes of insertion sort. For each pass write: the key value, the elements that were shifted, and the state of the array after insertion. Implement the sort and verify your trace.

Exercise 4
[**Insertion Sort — Binary Search Variant**] The inner `while` loop in insertion sort uses *linear search* to find the insertion position, performing both comparisons and shifts. Implement **Binary Insertion Sort**:

- Use `std::lower_bound` to find the correct insertion position in $O(\log n)$ comparisons.
- Still shift elements to make room — so shifts remain $O(n^2)$.

Count and print comparisons for both the standard and binary versions on the same array of size 20. Explain why the overall complexity is still $O(n^2)$.

Exercise 5
[**Counting Operations**] Write a program that runs both selection sort and insertion sort on the following three inputs of size $n = 15$:

- Already sorted: 1, 2, ..., 15.
- Reverse sorted: 15, 14, ..., 1.
- Random: your own choice of 15 distinct integers.

For each run, count and print the number of **comparisons** and **swaps/shifts**. Fill in the table below and explain whether the empirical results match the theoretical predictions.

Input	Sel cmp	Sel swaps	Ins cmp	Ins shifts
Sorted (best)				
Reversed (worst)				
Random (average)				

Exercise 6

[**Stability Test**] Selection sort is **not stable**. Demonstrate this with a concrete example:

- Define a `struct Card { int value; char suit; }`.
- Create an array with two cards of equal value but different suits: e.g. (5,H), (3,D), (5,D), (1,C).
- Run standard selection sort (by `value`).
- Show that the two 5s may have swapped their original order (not stable).
- Then implement a **stable selection sort** (shift instead of swap) and show that the relative order is now preserved.

Exercise 7

[**Generic Sort with Custom Comparator**] Write a single generic function:

```
template <typename T, typename Compare>
void insertionSort(vector<T>& arr, Compare cmp);
```

Use it to sort:

- i. A `vector<int>` in ascending order.
- ii. A `vector<string>` in descending alphabetical order.
- iii. A `vector<pair<int,string>` by the integer key ascending, breaking ties by the string alphabetically.

Exercise 8

[**Nearly Sorted Performance**] Insertion sort shines on nearly sorted data. Measure this empirically:

- Generate a sorted vector of size $n = 1000$.
- Introduce k random swaps to “disturb” it, for $k \in \{0, 5, 10, 50, 100, 500\}$.
- For each value of k , count the comparisons used by insertion sort and by selection sort.
- Print a table. Describe the trend: how quickly does insertion sort degrade as k increases?

Exercise 9

[**Sorting Structs**] Define a `struct Student { string name; double gpa; int year; }`. Implement both selection sort and insertion sort (as free functions) that sort a `vector<Student>` by:

- i. GPA descending (highest first).
- ii. Year ascending, then GPA descending (two-key sort).
- iii. Name alphabetically ascending.

For each ordering, print the sorted list. Verify that insertion sort’s result is **stable** (students with identical keys stay in original order).

Exercise 10

[**Hybrid Sort: Insertion + Selection**] Real-world sorters use hybrids. Implement and benchmark a **Hybrid Sort** that:

- Uses **insertion sort** when the subarray size $\leq$ threshold k .
- Uses **selection sort** otherwise, recursively splitting the unsorted region (each pass places one element, then recurse).

Test on arrays of size $n \in \{50, 200, 1000\}$ with threshold $k \in \{5, 10, 20\}$. Print comparison counts for each combination and identify the threshold that minimises comparisons. Explain your findings.
